MSPLIT End-to-End Writeup

From LightGBM Binning (`max_bin = 1024`) to Reference-Guided MSPLIT Training

Generated from the active implementation in this repository

April 14, 2026

Contents

1 Executive Overview **3**
- 1.1 One-Sentence Summary . . . 3
- 1.2 Why the Pipeline is Split in Two . . . 3
- 1.3 At a Glance: Stage A vs. Stage B . . . 4

2 Notation **4**

3 The End-to-End Pipeline in Practice **4**

4 Part I: LightGBM Training with `max_bin = 1024` **5**
- 4.1 Goal of the LightGBM Stage . . . 5
- 4.2 Dataset Split and Preprocessing . . . 5
- 4.3 Training the Teacher . . . 6
- 4.4 Extracting Split Thresholds . . . 6
- 4.5 From Thresholds to Integer Bins . . . 6
- 4.6 Teacher Logits . . . 7
- 4.7 Teacher Boundary Priors . . . 7
- 4.8 LightGBM Example . . . 7
- 4.9 What the LightGBM Stage Hands to MSPLIT . . . 8

5 Part II: MSPLIT Training **8**
- 5.1 What MSPLIT is Solving . . . 8
- 5.2 Leaf Objective . . . 8
- 5.3 Impurity Used Inside Candidate Search . . . 9
- 5.4 Hard vs. Soft Quantities . . . 9
- 5.5 Multiway Split Semantics . . . 10

6 How a Single MSPLIT Node is Solved **10**
- 6.1 Step 1: Compute Node Statistics . . . 10
- 6.2 Step 2: Build Ordered Occupied Bins . . . 10
- 6.3 Step 3: Build Atoms . . . 10
- 6.4 Atomic Example . . . 11
- 6.5 Step 4: Optional Atom Compression into Blocks . . . 11
- 6.6 Compression Example . . . 11
- 6.7 Step 5: Generate Two Candidate Families . . . 12

6.8 Candidate Scores . . . 12
6.9 Boundary Penalty . . . 13
6.10 Step 6: Compare the Two Families . . . 13
6.11 Step 7: Teacher-Guided Pair Pruning . . . 13
6.12 Step 8: Materialize Nominees . . . 13
6.13 Step 9: Exact Lower Bounds via Canonical Signatures . . . 14
6.14 Step 10: Exactify Only a Prefix . . . 14
6.15 Step 11: Recursive Solve or Greedy Completion . . . 14
6.16 Step 12: Keep the Split Only if It Beats the Leaf . . . 15

7 Local Partition Refinement (LPR) 15
7.1 What This Part Does . . . 15
7.2 Why Reassignment is Helpful . . . 15
7.3 The Three Types of Refinement Moves . . . 15
7.4 The Local Move Score . . . 16
7.5 Intuition for Components . . . 16
7.6 Simple Reassignment Example . . . 16
7.7 More Intuitive Example: Descent vs. Bridge vs. Cleanup . . . 17
7.8 Block Space First, Atom Space Second . . . 17

8 Reference Guidance and "Reference Ensemble Pruning" 17
8.1 What "Reference Ensemble" Means Here . . . 17
8.2 Soft Teacher Structure . . . 18
8.3 Reference-Floor Pruning . . . 18
8.4 Boundary-Aware Pruning . . . 18
8.5 Why This is Powerful . . . 18

9 A Worked End-to-End Example 18
9.1 Tiny One-Feature Example . . . 18
9.2 Stage A: LightGBM Binning . . . 19
9.3 Why This is a Good Example . . . 19
9.4 Stage B: MSPLIT Search . . . 19
9.5 What the Example Teaches . . . 19
9.6 Canonical Signature Lower-Bound Example . . . 20

10 How to Read the Full Algorithm Intuitively 20
10.1 The Most Useful Mental Model . . . 20
10.2 Why the Algorithm Can Stay Fast . . . 20
10.3 Why the Algorithm Can Stay Interpretable . . . 20

11 Inference 21

12 Practical Takeaways 21
12.1 What Each Major Mechanism is Really For . . . 21
12.2 If You Remember Only Five Things . . . 21

13 Conclusion 21

1 Executive Overview

The repository implements a two-stage training pipeline:

1. Train a LightGBM teacher with `max_bin = 1024` on the preprocessed fit split.
2. Extract the teacher's split thresholds, logits, and boundary-strength signals.
3. Convert each feature into an integer-binned feature.
4. Train MSPLIT on those integer bins, using the teacher only as guidance for search, pruning, and boundary shaping.

The important conceptual point is that *LightGBM is not the final model.* It is a high-resolution, fast, expressive teacher whose job is to tell MSPLIT:

- where useful feature boundaries probably are,
- what the teacher believes at each training row,
- which boundaries look strong enough to preserve, and
- which candidate multiway splits are worth exact recursive evaluation.

MSPLIT then solves a different problem: it searches for a small, interpretable, regularized multiway tree over the integer bins. At each node it compares leaving the node as a leaf against splitting on one feature into several groups of bins. The solver mixes exact recursion near the root with a cheaper greedy completion below a lookahead horizon.

1.1 One-Sentence Summary

LightGBM gives MSPLIT a good discrete language for the data, and MSPLIT uses that language to build a smaller, more interpretable tree while borrowing the teacher only as a guide, not as the final predictor.

1.2 Why the Pipeline is Split in Two

If we tried to search directly over raw continuous thresholds, the combinatorial search would be much larger. If we only used LightGBM, we would get a high-performing boosted tree ensemble, but not the kind of sparse, explicit, human-readable tree this repository is targeting.

The split pipeline gets the best of both worlds:

- **LightGBM stage**: quickly finds good cut points and produces rich guidance.
- **MSPLIT stage**: performs structured tree search in a much smaller, discrete space.

1.3 At a Glance: Stage A vs. Stage B

Aspect	LightGBM training	MSPLIT training
Primary role	Build a strong teacher and a feature-wise discretization	Build the final interpretable tree
Input	Preprocessed continuous features	Integer bins plus teacher guidance
Main output	Bin edges, teacher logits, boundary priors	Multiway decision tree
Optimization style	Gradient boosting with histogram bins	Recursive search with shortlist pruning and exactification
What gets reused downstream	Thresholds, logits, boundary strengths	The final tree itself

2 Notation

Let the training data at a node or dataset level be

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n},$$

where $x_i \in \mathbb{R}^p$ after preprocessing and $y_i \in \{0, 1, \ldots, C-1\}$.

The implementation also supports sample weights $w_i \geq 0$. In the benchmark workflow the weights are uniform and normalized so that

$$\sum_{i=1}^{n} w_i = 1.$$

After the LightGBM stage, each continuous feature x_{ij} is converted into an integer bin

$$z_{ij} \in \{0, 1, \ldots, B_j - 1\},$$

so each row becomes a vector z_i of integers.

We will use the following derived quantities often:

- $W_c(S) = \sum_{i \in S, y_i = c} w_i$: empirical class weight of subset S.
- $T_c(S) = \sum_{i \in S} w_i p_{ic}^{(\text{teacher})}$: teacher soft class weight of subset S.
- $R(S)$: weighted disagreement between teacher hard predictions and labels on S.

For binary classification we abbreviate $W_+(S)$, $W_-(S)$ and $T_+(S)$, $T_-(S)$.

3 The End-to-End Pipeline in Practice

The benchmark cache builder in this repository follows the same high-level sequence every time:

1. Load a named dataset.
2. Encode labels numerically.
3. Split rows into fit, validation, and test subsets.

4. Fit the preprocessing pipeline on the fit split.
5. Transform fit, validation, and test into dense numeric arrays.
6. Fit the LightGBM binner on the fit split, with validation for early stopping.
7. Produce integer-binned matrices $Z_{\text{fit}}, Z_{\text{val}}, Z_{\text{test}}$.
8. Save teacher logits and teacher boundary tensors.
9. Fit MSPLIT on Z_{fit}, y_{fit}, and the saved teacher guidance.
10. Evaluate the resulting tree on the cached validation and test bins.

The repository's maintained benchmark path uses exactly this idea: first build or reuse the LightGBM cache, then call the native MSPLIT solver on the cached integer bins.

4 Part I: LightGBM Training with `max_bin = 1024`

4.1 Goal of the LightGBM Stage

The LightGBM stage has three jobs:

1. discretize each feature into an ordered set of bins,
2. produce per-row teacher logits or class probabilities,
3. produce per-boundary signals telling MSPLIT which feature boundaries look important.

This stage is a *teacher-construction and representation-construction step*. It is not the final model selection step.

4.2 Dataset Split and Preprocessing

Before LightGBM is trained, the pipeline creates fit, validation, and test splits. The default cache builder uses:

- test size = 0.2,
- validation size = 0.1,
- fit size = 0.7.

The fit split is used to fit the preprocessing pipeline. Validation and test are transformed using the same fitted preprocessor.

So LightGBM does *not* see raw repository datasets directly. It sees preprocessed numeric features. That matters because the extracted bin boundaries are boundaries in the preprocessed feature space.

4.3 Training the Teacher

The binner fits a LightGBM classifier with `max_bin = 1024`. Conceptually:

$$\texttt{teacher} = \mathrm{LightGBM}(X_{\mathrm{fit}}, y_{\mathrm{fit}}; \texttt{max_bin} = 1024).$$

In the default benchmark cache settings, the teacher also uses:

- many boosting rounds with early stopping,
- large enough leaves to provide a sharp teacher,
- teacher logit collection enabled,
- CPU mode by default, with GPU support available in the helper.

The code also supports multiple teacher runs:

$$f^{(\mathrm{teacher})}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x).$$

When $M > 1$, the threshold extraction and the raw margins are aggregated across runs. In that sense the repository can use a small *reference ensemble*, even though many benchmark runs use $M = 1$.

4.4 Extracting Split Thresholds

After fitting the teacher, the implementation dumps the LightGBM trees and walks every internal split. For each feature j, it collects thresholds τ together with their accumulated split gain:

$$\mathrm{Score}_j(\tau) = \sum_{\text{teacher splits on feature } j \text{ at } \tau} \max(\mathrm{split_gain}, 10^{-12}).$$

Then it keeps the highest-scoring thresholds for each feature, sorted back into increasing order. Because `max_bin = 1024`, the number of edges per feature is capped at

$$\texttt{max_edges} = 1024 - 1 = 1023.$$

If a feature receives no usable teacher thresholds, the implementation falls back to quantile edges. This is important: the pipeline is robust even when the teacher barely uses a feature.

4.5 From Thresholds to Integer Bins

If feature j ends up with ordered edges

$$E_j = (e_{j1}, e_{j2}, \ldots, e_{j,K_j}),$$

then the integer bin is

$$z_{ij} = \mathrm{digitize}(x_{ij}; E_j).$$

Equivalently, in scalar form:

$$z_{ij} = \sum_{k=1}^{K_j} \mathbf{1}\{x_{ij} \geq e_{jk}\},$$

with bins numbered from 0 through K_j.

So each feature becomes a small ordered alphabet. MSPLIT never searches raw floating-point thresholds directly; it only searches groupings of these integer bins.

4.6 Teacher Logits

The binner also stores the teacher's raw margin for every fit row. For binary classification:

$$\ell_i = f^{(\text{teacher})}(x_i), \qquad p_i = \sigma(\ell_i) = \frac{1}{1+e^{-\ell_i}}.$$

For multiclass classification, the teacher yields a vector of logits and the probabilities come from softmax:

$$p_{ic} = \frac{e^{\ell_{ic}}}{\sum_{r=1}^{C} e^{\ell_{ir}}}.$$

MSPLIT uses these teacher probabilities later to compute *soft impurity* and soft class totals. This is a key design choice: a bin assignment is not evaluated only by the ground-truth labels, but also by whether it respects the teacher's soft structure.

4.7 Teacher Boundary Priors

Beyond per-row logits, the LightGBM stage computes three per-feature, per-boundary signals:

- **Boundary gain**: how much split gain the teacher accumulated at that boundary.
- **Boundary cover**: how many samples the teacher split there.
- **Boundary value jump**: how different the left and right subtree leaf values were.

For a boundary b on feature j, you can think of these as:

$$G_{jb}, \qquad C_{jb}, \qquad V_{jb}.$$

The solver normalizes them using $\log(1+\cdot)$, rescales each channel by its feature-wise maximum, and averages the three normalized channels:

$$S_{jb} = \frac{1}{3}\left(\frac{\log(1+G_{jb})}{\max_u \log(1+G_{ju})} + \frac{\log(1+C_{jb})}{\max_u \log(1+C_{ju})} + \frac{\log(1+V_{jb})}{\max_u \log(1+V_{ju})}\right).$$

The solver then stores prefix sums of these strengths so it can answer queries of the form:

$$\text{BoundaryStrength}(j,a,b) = \sum_{u=a}^{b-1} S_{ju}.$$

Intuition: if the teacher repeatedly splits at a boundary, does so on many rows, and creates a large predictive jump there, MSPLIT should be reluctant to create a noncontiguous grouping that leaps across that boundary.

4.8 LightGBM Example

Suppose one preprocessed feature is "age-like" and the teacher finds strong thresholds at 10, 20, and 35. Then

$$E = (10, 20, 35),$$

which induces four bins:

$$(-\infty, 10),\ [10, 20),\ [20, 35),\ [35, \infty).$$

If a row has feature value 24, its integer bin is 2. If another row has value 7, its bin is 0.

Now imagine the teacher strongly prefers the boundary at 20. MSPLIT will still be allowed to group bins on either side of that boundary together, but doing so will face a stronger boundary penalty and stronger competition from candidates that preserve the teacher's structure there.

4.9 What the LightGBM Stage Hands to MSPLIT

At the end of the first stage, MSPLIT receives:

- Z: integer bins for each row and feature,
- teacher logits or probabilities,
- teacher boundary gain,
- teacher boundary cover,
- teacher boundary value jump.

This is the bridge between the two stages. Once these artifacts exist, the second stage is a discrete structured search problem.

5 Part II: MSPLIT Training

5.1 What MSPLIT is Solving

MSPLIT searches for a regularized tree over integer bins. Each internal node chooses:

- one feature j,
- a number of groups $g \geq 2$,
- an assignment of that feature's occupied bins into g groups,
- one child subtree per group.

The implementation allows groups to be unions of ordered spans:

$$S_{j,r} = \bigcup_{m=1}^{M_r} [a_{rm}, b_{rm}],$$

where each $S_{j,r}$ is the set of bin ids routed to child r.

If $M_r > 1$, the group is noncontiguous. This is one of the major differences from a standard binary threshold tree: MSPLIT can represent patterns such as "low or high values go left, middle values go right".

5.2 Leaf Objective

At any node, MSPLIT compares splitting against staying as a leaf.

For a subset S, the empirical leaf loss is the weighted misclassification of predicting the majority class in that subset:

Binary case.

$$L_{\text{leaf}}(S) = \min\{W_+(S), W_-(S)\}.$$

Multiclass case.

$$L_{\text{leaf}}(S) = \sum_{c=1}^{C} W_c(S) - \max_c W_c(S).$$

The solver adds a regularization term λ per leaf:

$$\text{LeafObjective}(S) = L_{\text{leaf}}(S) + \lambda.$$

So the total tree objective is

$$\text{Obj}(T) = \sum_{\ell \in \text{Leaves}(T)} \left(L_{\text{leaf}}(\ell) + \lambda \right).$$

This means every extra leaf must earn its keep. A split is only worthwhile if the reduction in weighted error is large enough to pay for the added leaf regularization.

5.3 Impurity Used Inside Candidate Search

MSPLIT uses impurity-style scores to generate strong candidate splits.

For binary class weights (a, b):

$$I(a, b) = (a + b) - \frac{a^2 + b^2}{a + b}.$$

For multiclass weights $(u_1, \ldots, u_C)$:

$$I(u_1, \ldots, u_C) = \sum_{c=1}^{C} u_c - \frac{\sum_{c=1}^{C} u_c^2}{\sum_{c=1}^{C} u_c}.$$

This is essentially weighted Gini times the node weight. The code uses the same formula on:

- **hard weights**: true label counts,
- **soft teacher weights**: teacher probability mass.

5.4 Hard vs. Soft Quantities

For any candidate grouping, the solver tracks both empirical and teacher-based summaries:

- **Hard loss**: misclassification computed from true labels.
- **Soft loss**: the same formula but using teacher probability mass.
- **Hard impurity**: impurity on empirical labels.
- **Soft impurity**: impurity on teacher probabilities.

This matters because two candidate splits can have similar hard error but very different teacher consistency.

Why soft impurity helps intuition. Suppose two bins both predict class 1 under the hard majority rule. If the teacher is 0.51 positive in one bin and 0.99 positive in the other, the teacher is telling us those bins are not equally similar. Soft impurity preserves that distinction.

5.5 Multiway Split Semantics

At a node, MSPLIT does not necessarily ask "what is the best threshold?" It asks:

> "For this feature's occupied bins, how should we partition the bins into a small number of groups so that each child becomes easier to classify?"

That is a more flexible question. A threshold tree can only create contiguous left/right partitions. MSPLIT can create:

- contiguous multiway segments,
- or, when it helps enough, noncontiguous unions of spans.

6 How a Single MSPLIT Node is Solved

6.1 Step 1: Compute Node Statistics

For the current subset S, the solver computes:

- empirical class weights,
- whether the node is pure,
- leaf objective,
- teacher disagreement weight

$$R(S) = \sum_{i \in S} w_i \mathbf{1}\{\hat{y}_i^{(\text{teacher})} \neq y_i\},$$

- weight totals used for complexity scaling.

If the node is pure, too small, or depth-limited, the solver stops immediately and returns a leaf.

6.2 Step 2: Build Ordered Occupied Bins

For each feature j, the node only cares about bins that are actually occupied inside the node. If feature j globally has many possible bins but only a few appear in this subset, the local search only uses the occupied ones.

This is a major computational win. Near the leaves, the effective feature alphabet is often much smaller than the global `max_bin` budget.

6.3 Step 3: Build Atoms

An **atom** is the smallest local unit of feature structure at the node: typically one occupied bin, with both empirical and teacher summaries attached.

For atom a, MSPLIT stores quantities like:

- row count,
- empirical class weights,

- teacher class weights,
- empirical majority label,
- teacher majority label.

So each atom already knows both "what the data says" and "what the teacher says".

6.4 Atomic Example

Suppose a feature has occupied bins $0, 1, 2, 3$, and at the current node the rows look like:

Bin	True majority	Teacher majority
0	0	0
1	0	0
2	1	1
3	1	1

Then bins 0 and 1 are very compatible, and bins 2 and 3 are very compatible. A good split probably preserves that local structure.

6.5 Step 4: Optional Atom Compression into Blocks

The active implementation can compress adjacent atoms into larger **blocks**. Under the default rule, two neighboring atoms are merged if they have:

- the same empirical prediction, and
- the same teacher prediction.

Intuition: if both the labels and the teacher agree that two neighboring bins behave the same way, there is little value in treating them as separate units during early candidate generation.

This gives a coarse representation that is faster to search.

6.6 Compression Example

Suppose bins $0, 1, 2, 3, 4, 5$ have:

Bin	Empirical prediction	Teacher prediction
0	0	0
1	0	0
2	0	0
3	1	1
4	1	1
5	1	1

Then the search can safely start from two blocks:

$$\{0, 1, 2\}, \qquad \{3, 4, 5\}.$$

This does *not* mean the final split is forced to keep exactly those blocks. It means the initial coarse search starts there and later lifts back to atom-level refinement.

6.7 Step 5: Generate Two Candidate Families

For each feature and for each arity g (number of groups), the solver generates two family nominees:

1. an **impurity family** candidate,
2. a **hard-loss family** candidate.

These are not redundant. The impurity family tries to produce children that are internally pure and teacher-consistent. The hard-loss family aims more directly at classification error.

The active solver explicitly compares them and may keep one or both.

6.8 Candidate Scores

For a candidate grouping, the solver aggregates:

- hard loss,
- soft loss,
- hard impurity,
- soft impurity,
- boundary penalty,
- number of connected components across groups.

For impurity-mode candidate generation, the primary objective is

$$J_{\text{imp}} = I_{\text{hard}} + I_{\text{soft}}.$$

For hard-loss mode, the primary objective is

$$J_{\text{hard}} = L_{\text{hard}}.$$

The cheap shortlist proxy then adds a complexity term:

$$\widetilde{J} = J_{\text{primary}} + \mu_{\text{node}} \left(\text{BoundaryPenalty} + \text{Components}\right).$$

The scaling factor is

$$\mu_{\text{node}} = \frac{1}{n_{\text{eff}}}, \qquad n_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2},$$

with the uniform-weight case reducing to roughly $1/|S|$.

Intuition. Large nodes get a smaller complexity multiplier, so the solver is willing to tolerate more structural complexity near the root. Small nodes get a larger multiplier, so extra disconnected pieces become more expensive deeper in the tree.

6.9 Boundary Penalty

If one group contains atoms that are separated by a gap, the candidate pays a noncontiguous boundary penalty:

$$\text{PenaltyGap} = \frac{\text{BoundaryStrength}(j, z_{\text{left}}, z_{\text{right}})}{z_{\text{right}} - z_{\text{left}}}.$$

So if a group jumps across a strong teacher boundary, that jump costs more than jumping across a weak boundary.

Interpretation. The teacher is saying: "I think something real happens here." MSPLIT is still allowed to ignore that advice, but it must justify doing so by enough gains elsewhere.

6.10 Step 6: Compare the Two Families

The solver does not blindly keep both impurity and hard-loss candidates. It compares them:

- if they are effectively equivalent, keep one;
- if one dominates, keep the winner;
- if they disagree in a meaningful way, keep both.

This is useful because hard loss and impurity can disagree, especially when the node is small or when the teacher sees structure not yet visible in hard majority counts.

6.11 Step 7: Teacher-Guided Pair Pruning

Once feature-arity pairs have been generated cheaply, the solver uses a teacher-guided baseline:

$$F_{\text{ref}}(g) = R(S) + \lambda g.$$

Here $R(S)$ is the node-level weighted disagreement between teacher hard predictions and true labels, and g is the candidate's number of groups. In the implementation this acts as a *reference floor-like pruning score* for shortlist management.

Pairs with poor reference behavior relative to the current search upper bound are discarded early.

Above the lookahead horizon, the solver keeps only a prefix of the surviving pairs:

$$\text{pair budget} = \begin{cases} \min(K, N), & \text{if exactify_top_k} = K > 0, \\ \lceil \sqrt{N} \rceil, & \text{otherwise.} \end{cases}$$

This is one of the key ideas of the algorithm: many pairs are scored cheaply, very few are allowed to trigger expensive recursive evaluation.

6.12 Step 8: Materialize Nominees

For each shortlisted nominee, the solver expands the feature assignment into actual child subsets:

- which rows go to child 1,
- which rows go to child 2,

- and so on.

At this point the solver knows the exact row set of every child. Now it can compute child statistics and stronger bounds.

6.13 Step 9: Exact Lower Bounds via Canonical Signatures

Once a candidate's children are known, the solver can compute a real structural lower bound using *canonical signature blocks*. The idea is simple:

> If two rows are identical in the current binned feature representation, no future subtree can ever separate them.

So for each child, rows are grouped by identical full bin-signatures across all features. For each such signature block, the solver adds the minimum possible leaf loss inside that block. Summing those block losses gives a valid lower bound.

Binary intuition. If a signature block contains one positive row and one negative row, then no descendant split can separate them, because they are identical on every remaining feature. That block contributes at least one weighted mistake.

Formally, if $\mathcal{B}$ is the set of signature blocks,

$$\text{SignatureBound}(S) = \lambda + \sum_{B \in \mathcal{B}} \min\{W_+(B), W_-(B)\}$$

in the binary case, with the multiclass analogue replacing the binary leaf loss by the multiclass majority-vote error.

6.14 Step 10: Exactify Only a Prefix

Above the lookahead horizon, the solver exactifies only a shortlist prefix. It also preserves strong anchors:

- the best impurity-side nominee,
- the best hard-loss-side nominee.

So even if the shortlist cap is K, the realized exactification prefix can be larger than K if those anchors differ.

This is a subtle but important design choice: the solver saves work without collapsing too aggressively onto a single scoring philosophy.

6.15 Step 11: Recursive Solve or Greedy Completion

When a nominee is exactified, each child is solved recursively.

- **Above lookahead depth**: recurse in exact mode, again with shortlist generation and pruning.
- **At or below lookahead depth**: switch to a cheaper greedy completion strategy.

The default effective lookahead depth is roughly half the full depth budget:

$$d_{\text{lookahead}} = \max\left(1, \left\lfloor \frac{D+1}{2} \right\rfloor\right).$$

This means the top of the tree gets most of the expensive search budget, which is where global structural decisions matter most.

6.16 Step 12: Keep the Split Only if It Beats the Leaf

Even after all that work, a split is not automatically accepted. The solver compares the best split found to the plain leaf objective at the current node:

$$\text{accept split if BestSplitObjective} < \text{LeafObjective}(S).$$

If not, the node stays a leaf.

This final check is conceptually important. Candidate generation may suggest many plausible partitions, but the solver only commits to one if the full downstream objective really improves.

7 Local Partition Refinement (LPR)

7.1 What This Part Does

The active nonlinear solver uses a local refinement routine, which we call *Local Partition Refinement (LPR)* and implement in `refine_atomized_candidate_partition_locally`.

Its job is:

> Start from a reasonable coarse grouping of bins, then improve it by moving contiguous blocks of adjacent occupied bins from one group to another.

This is where the search becomes intuitive: instead of rebuilding a split from scratch, the algorithm asks whether specific pieces of the current grouping should be reassigned.

7.2 Why Reassignment is Helpful

The coarse seed candidate is fast to compute, but it may not be locally optimal. Maybe:

- one small component clearly belongs to another group,
- a noncontiguous detour should be removed,
- or a tie should be broken in the teacher's favor.

Local reassignment lets the solver fix those issues cheaply.

7.3 The Three Types of Refinement Moves

The implementation effectively reasons about three kinds of moves.

1. Descent moves. These strictly improve the primary objective of the current mode.

If the current mode is impurity mode, the primary gain is the drop in

$$I_{\text{hard}} + I_{\text{soft}}.$$

If the current mode is hard-loss mode, the primary gain is the drop in

$$L_{\text{hard}}.$$

2. Bridge moves. These are near-zero-primary-gain moves that still improve the guide objective enough to help the search escape a plateau. The implementation is careful with these and only allows limited usage.

3. Cleanup moves. These keep the primary objective unchanged but reduce structural complexity, especially the number of disconnected components. They are how the solver simplifies awkward groupings after the main score has stabilized.

7.4 The Local Move Score

For a proposed local reassignment move, the implementation tracks a score of the form

$$\Delta J = \Delta_{\text{primary}} - \mu_{\text{node}}\, \Delta_{\text{components}}.$$

Here:

- Δ_{primary} is the improvement in the active primary objective,
- $\Delta_{\text{components}}$ is the change in the number of connected pieces.

So a move that decreases the number of components is rewarded, and a move that fragments a group into more pieces is penalized.

7.5 Intuition for Components

If one group is $\{0, 1, 2, 7, 8\}$, it has two connected components:

$$\{0, 1, 2\}, \qquad \{7, 8\}.$$

Groups with many disconnected pieces are harder to explain to humans and more likely to be overfitting strange local behavior. LPR pushes the solution back toward simpler groupings unless the primary objective strongly justifies the extra fragmentation.

7.6 Simple Reassignment Example

Suppose a feature currently has group assignment

$$G_1 = \{0, 1, 4\}, \qquad G_2 = \{2, 3\}.$$

Then G_1 has two connected components: $\{0, 1\}$ and $\{4\}$.

If moving bin 4 from G_1 to G_2 leaves the classification loss unchanged, then the new grouping

$$G_1' = \{0, 1\}, \qquad G_2' = \{2, 3, 4\}$$

has fewer total components. A cleanup move should prefer the new grouping because it is equally predictive but structurally cleaner.

7.7 More Intuitive Example: Descent vs. Bridge vs. Cleanup

Imagine the current grouping is

$$G_1 = \{0, 1, 2\}, \qquad G_2 = \{3, 4, 5\},$$

but bin 2 behaves much more like bins 3 and 4 than like bins 0 and 1.

- If moving bin 2 from G_1 to G_2 lowers the main objective, that is a **descent move**.
- If the main objective stays flat but the teacher-consistency score improves, that is a **bridge move**.
- If the main objective stays flat and the real gain is that the grouping becomes more contiguous, that is a **cleanup move**.

This is a very useful way to read the algorithm: it is not just searching for the best split, it is also editing candidate splits into cleaner versions of themselves.

7.8 Block Space First, Atom Space Second

The active implementation often follows this pattern:

1. build a coarse candidate in compressed block space,
2. refine it in block space,
3. lift it back to atom space,
4. refine it again in atom space.

This is a smart compromise. Coarse search is cheaper, while atom-level refinement recovers fine details if they matter.

8 Reference Guidance and "Reference Ensemble Pruning"

8.1 What "Reference Ensemble" Means Here

The user-facing intuition is that the first stage may provide a *reference teacher*, possibly from one LightGBM model or a small average of several teacher runs. MSPLIT never tries to become that ensemble. Instead, it uses the ensemble's outputs as guidance.

There are three main ways this guidance enters:

1. through teacher probabilities in soft impurity,
2. through teacher disagreement in reference pruning,
3. through teacher-derived boundary strengths in noncontiguous penalties.

8.2 Soft Teacher Structure

Teacher probabilities create soft class totals:

$$T_c(S) = \sum_{i \in S} w_i p_{ic}^{(\text{teacher})}.$$

Those totals feed the soft impurity and soft loss calculations. So candidates are rewarded when they not only fit the true labels but also organize the data in a way that the teacher sees as internally coherent.

8.3 Reference-Floor Pruning

The solver's shortlist logic uses the teacher disagreement baseline

$$F_{\text{ref}}(g) = R(S) + \lambda g$$

to rank and prune candidate pairs and nominee candidates.

The right intuition is:

> If the teacher is already making only a small amount of error on this node, then a candidate split had better justify its extra groups and leaves. If it cannot compete even against a teacher-guided baseline, it should not consume expensive exact recursive search.

8.4 Boundary-Aware Pruning

Teacher-derived boundary strengths make it expensive to create unnatural noncontiguous groups that jump across strong boundaries. This has a pruning effect even before recursion, because awkward candidates receive worse cheap proxy scores and are less likely to survive into the exactification prefix.

8.5 Why This is Powerful

Without teacher guidance, MSPLIT would still be a discrete tree searcher, but it would need to treat many more candidate splits as plausible. Teacher guidance turns the search from:

> "Which of these thousands of feature-group assignments might be good?"

into

> "Which of these teacher-compatible assignments are worth exact evaluation, and when is it worth deviating from the teacher?"

9 A Worked End-to-End Example

9.1 Tiny One-Feature Example

Consider one preprocessed feature with values:

$$2,\ 4,\ 8,\ 11,\ 14,\ 17,\ 22,\ 29$$

and binary labels:

$$0,\ 0,\ 0,\ 1,\ 1,\ 1,\ 0,\ 0.$$

9.2 Stage A: LightGBM Binning

Suppose the teacher extracts thresholds near 10 and 20. Then the feature becomes three bins:

- bin 0: values < 10,
- bin 1: values in $[10, 20)$,
- bin 2: values ≥ 20.

The binned labels are now approximately:

Bin	Labels inside bin
0	$0, 0, 0$
1	$1, 1, 1$
2	$0, 0$

9.3 Why This is a Good Example

The pattern is "low is 0, middle is 1, high is 0". A plain binary threshold split cannot express that in one node. A multiway split can.

9.4 Stage B: MSPLIT Search

MSPLIT now has a few obvious options:

1. keep a single leaf,
2. make a 2-group split like $\{0, 1\}$ vs. $\{2\}$,
3. make a 2-group noncontiguous split like $\{0, 2\}$ vs. $\{1\}$,
4. make a 3-group split $\{0\}$, $\{1\}$, $\{2\}$.

If the teacher strongly believes the boundaries at both 10 and 20 are real, then the noncontiguous option $\{0, 2\}$ vs. $\{1\}$ gets penalized for leaping across a strong middle region. The 3-way split may therefore win, even though the noncontiguous 2-group split is expressive.

9.5 What the Example Teaches

This tiny example captures the entire philosophy:

- LightGBM converts the feature into a small ordered alphabet.
- MSPLIT searches over groupings of that alphabet.
- Teacher guidance tells the search which groupings look natural.
- Regularization decides whether the extra groups are worth paying for.

9.6 Canonical Signature Lower-Bound Example

Now suppose inside one candidate child there are two rows that have the exact same full binned vector across *all* features, but opposite labels.

Then no descendant subtree can ever separate them. No matter what future splits are chosen, at least one of those rows must be misclassified. That unavoidable error becomes part of the child's signature lower bound.

This is why exactification can prune candidates before finishing full recursion: some candidates are already doomed by identical-feature collisions.

10 How to Read the Full Algorithm Intuitively

10.1 The Most Useful Mental Model

The easiest mental model is:

1. LightGBM says where the data likely bends.
2. MSPLIT turns those bends into discrete bins.
3. For each node, MSPLIT tries a small number of ways to group those bins.
4. It refines awkward groupings by moving contiguous pieces around.
5. It uses the teacher to avoid wasting time on implausible candidates.
6. It only commits to a split if the full downstream objective beats a leaf.

10.2 Why the Algorithm Can Stay Fast

The solver stays practical because it does *not* exactify every possible split.

It relies on:

- binning to shrink the search space,
- atom compression to coarsen local structure,
- impurity and hard-loss families to propose strong seeds,
- teacher-based shortlist pruning,
- canonical-signature lower bounds,
- greedy completion below the lookahead horizon.

All of these pieces are necessary. Remove one, and the search either becomes more expensive or less selective.

10.3 Why the Algorithm Can Stay Interpretable

The final model is still a tree whose nodes say things like:

> "Look at feature j. If its bin is in this set of spans, go to child 1; otherwise, if it is in that set of spans, go to child 2."

That is much more interpretable than a large boosted ensemble, while still being more expressive than a pure binary-threshold tree.

11 Inference

At prediction time, the full pipeline is:

1. preprocess the raw features using the cached preprocessor,
2. digitize each feature with the cached LightGBM edges,
3. traverse the MSPLIT tree using bin membership in the node's group spans.

If a bin does not match a stored span exactly at a node, the implementation falls back to the nearest group or a fallback prediction. This makes the tree robust to sparse local training support.

12 Practical Takeaways

12.1 What Each Major Mechanism is Really For

Mechanism	**Practical purpose**
LightGBM thresholds	Reduce continuous search to an ordered discrete search
Teacher logits	Provide soft structure beyond hard labels
Boundary priors	Discourage unnatural noncontiguous jumps across strong teacher boundaries
Atoms and blocks	Make local search cheaper and more structured
Impurity family	Finds partitions that create pure, teacher-consistent children
Hard-loss family	Finds partitions that directly reduce classification error
LPR	Repairs and simplifies seed candidates
Reference pruning	Prevents expensive recursion on weak candidates
Signature bounds	Give exact lower bounds once child subsets are known
Lookahead horizon	Concentrates expensive exact search near the root

12.2 If You Remember Only Five Things

1. LightGBM provides the bins, not the final tree.
2. MSPLIT searches over *groupings of bins*, not raw thresholds.
3. The solver tracks both hard label structure and soft teacher structure.
4. Block reassignment is a local editing step that makes candidates better and simpler.
5. Exact search is used selectively; pruning and bounds are what make the whole approach practical.

13 Conclusion

The algorithm in this repository is easiest to understand as a division of labor.

LightGBM is the fast continuous explorer. It tells us where useful feature boundaries are, what a strong teacher believes, and which boundaries are worth respecting. MSPLIT is the discrete

tree architect. It takes those high-resolution hints and searches for a smaller, regularized, human-readable multiway tree.

The sophisticated pieces such as atom compression, dual candidate families, LPR, reference-guided pruning, and signature-based lower bounds are not independent tricks. They are all different ways of answering the same question:

> How do we keep only the expensive recursive search that is likely to matter?

That is why the pipeline works end-to-end. The first stage makes the second stage possible, and the second stage turns the first stage's rich guidance into an interpretable final model.